

Expert Decision Replay Platform

Project Report — Milestone 1 & Milestone 2

Stefin C Siju | Jyothi Engineering College | Springboard Mentorship Program

September 2026

Contents

1. Introduction	1
2. Objectives	2
3. Technology Stack	2
4. System Architecture	2
5. Roles & Permissions	3
6. Milestone 1 — Foundation	3
6.1 Features delivered	3
6.2 Milestone 1 API endpoints	4
6.3 Data model	4
7. Milestone 2 — Decisions, Discussion & History	4
7.1 Features delivered	4
7.2 Milestone 2 API endpoints	5
7.3 Data model additions	5
7.4 Frontend	5
8. Testing	5
9. Screenshots	6
10. Project Structure	10
11. Conclusion & Next Steps	11

1. Introduction

The **Expert Decision Replay Platform (EDRP)** is a centralized web application for documenting organizational decisions, tracking the alternatives considered, recording discussion around them, and preserving a full audit history of how and why a

decision changed over time. It is built as part of the Springboard mentorship program, developed solo end-to-end across two milestones.

The core problem EDRP addresses is a common one in organizations: decisions get made in meetings and chat threads, the reasoning behind them is never written down, and six months later nobody can explain why a particular choice was made. EDRP gives teams a single place to record the context, the alternatives that were weighed, who approved what, and how the decision evolved.

2. Objectives

- Provide secure, role-based access to a decision-management workspace
- Let users document a decision's context, chosen option, and consequences
- Let users record and compare **alternatives** to a decision with cost, risk, and feasibility scoring
- Support **threaded discussion** on each decision
- Support **file attachments** as supporting evidence
- Maintain an **immutable version history** every time a decision is edited
- Enforce permissions so that editing and deleting are restricted appropriately by role

3. Technology Stack

Layer	Technology
Backend framework	FastAPI (Python)
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	JWT (python-jose), bcrypt password hashing
Frontend framework	React 18 (Vite)
Frontend routing	React Router v6
HTTP client	Native browser fetch, via a small custom wrapper in <code>api.js</code>
Testing	Pytest + FastAPI TestClient (in-memory SQLite)

4. System Architecture

EDRP follows a standard three-tier architecture:

1. **Frontend (React + Vite)** — a single-page application with client-side routing. Talks to the backend over REST via the browser's built-in `fetch`, wrapped in a single `request()` helper (`src/api.js`) that centralizes JSON handling, JWT attachment, and error parsing.

2. **Backend (FastAPI)** — exposes REST endpoints, enforces authentication and role-based permissions, and handles all business logic.
3. **Database (PostgreSQL)** — stores users, roles, teams, decisions, alternatives, comments, attachments, and version history.

Authentication uses JSON Web Tokens: on login, the backend issues a JWT containing the user's email and role, which the frontend stores and attaches to subsequent requests. The backend decodes and verifies this token on every protected route via a `get_current_user` dependency.

5. Roles & Permissions

Four roles exist system-wide, seeded automatically on backend startup:

Role	Typical permissions
Employee	Create decisions, comment, view everything, edit/delete only what they created
Reviewer	Everything an Employee can, plus edit any decision's status
Manager	Everything a Reviewer can, plus create teams and assign members
Administrator	Full access, including deleting decisions and managing user roles

Permission checks happen on the backend (not just hidden in the UI), so a request that bypasses the frontend is still correctly rejected. For example, deleting a decision is restricted to Administrators only — everyone else is expected to archive a decision instead of erasing it, to preserve institutional history.

6. Milestone 1 — Foundation

Scope: FastAPI + React project setup, authentication, and user/team management with role-based access.

6.1 Features delivered

- User registration and login with JWT-based authentication
- Role-based access control backed by a normalized roles table (not a hardcoded enum)
- User profile management (name, phone, department, designation, profile image)
- Admin capability to list all users and change their roles
- Team management: create teams, list teams, assign users to a team

6.2 Milestone 1 API endpoints

Method	Path	Access
POST	/register	Public
POST	/login	Public
GET	/me	Any logged-in user
PUT	/me	Any logged-in user
PUT	/me/profile	Any logged-in user
GET	/users	Administrator only
PUT	/users/{id}/role	Administrator only
POST	/teams	Administrator, Manager
GET	/teams	Any logged-in user
PUT	/teams/assign/{user_id}	Administrator, Manager

6.3 Data model

Four core tables: roles, teams, users, user_profiles. Full field-level detail and the entity-relationship diagram are in docs/database-design.md.

Key design decisions:

- Roles are a normalized table rather than a hardcoded enum, so a new role can be added later with a simple insert, no schema migration.
- UserProfile is a separate 1:1 table from User, keeping registration minimal — a user can sign up with just name, email, and password, and fill in optional details later.
- Passwords are never stored in plaintext, only as a bcrypt hash.

7. Milestone 2 — Decisions, Discussion & History

Scope: decision management, alternative comparison, file uploads, discussion module, version tracking.

7.1 Features delivered

- **Decisions** — create, list (filterable by category/status), view, update, and delete a decision, each with a problem statement, category, status, and rationale
- **Alternatives** — attach multiple alternatives to a decision, each scored on estimated cost, risk (0-10), and feasibility (0-10); the system automatically computes a composite score and flags the top-scoring alternative as recommended
- **Discussion (Comments)** — threaded comments on a decision, supporting replies via a parent_id self-reference
- **Attachments** — file upload and listing per decision, stored under static/uploads/ with filename, size, and content type recorded
- **Version tracking** — every update to a decision increments its version number and stores an immutable snapshot (title, problem statement, category, status, rationale, and a change summary) in a separate decision_versions table, so the full edit history can be replayed

7.2 Milestone 2 API endpoints

Method	Path	Access
POST	/decisions	Any logged-in user
GET	/decisions	Any logged-in user (filter by category/status)
GET	/decisions/{id}	Any logged-in user
PUT	/decisions/{id}	Creator, Reviewer, Manager, Administrator
DELETE	/decisions/{id}	Administrator only
POST	/decisions/{id}/alternatives	Any logged-in user
GET	/decisions/{id}/alternatives	Any logged-in user
POST	/decisions/{id}/comments	Any logged-in user
GET	/decisions/{id}/comments	Any logged-in user
DELETE	/decisions/{id}/comments/{comment_id}	Comment author or privileged role
POST	/decisions/{id}/upload	Any logged-in user
GET	/decisions/{id}/attachments	Any logged-in user
GET	/decisions/{id}/versions	Any logged-in user

7.3 Data model additions

Milestone 2 adds five tables on top of Milestone 1's schema: decisions, alternatives, comments, attachments, and decision_versions. Each links back to decisions.id, and decisions.created_by_id links to users.id, extending the Milestone 1 schema without altering it.

Notable design point: the alternative-scoring model computes a **composite score** as $\text{feasibility} - \text{risk} - (\text{estimated_cost} / 1000)$, favoring alternatives that are more feasible, lower risk, and lower cost. The alternative with the highest composite score for a decision is automatically flagged as the recommended option.

7.4 Frontend

The frontend was rebuilt around React Router with a persistent sidebar layout, replacing Milestone 1's single-page tab switcher (which no longer scaled once decision detail screens needed their own bookmarkable URLs). It includes:

- A Decisions list page with category/status filters and a "New Decision" modal
- A Decision Detail page with tabs for Overview, Alternatives, Comments, Attachments, and Versions
- Milestone 1's Profile, Teams, and Admin pages, restyled to match

8. Testing

The backend is covered by an automated Pytest suite using FastAPI's TestClient against an in-memory SQLite database, so tests run without any external database dependency.

Result: 15 / 15 tests passing.

Test file	Covers
test_milestone1.py	Root endpoint, registration, login, duplicate-email rejection, wrong-password rejection, profile updates, admin-only user listing, team creation & assignment
test_milestone2.py	Decision create/get, list & filter, alternatives & recommendation scoring, comment threads, employee blocked from editing another user's decision, manager can update a decision (and a version snapshot is recorded), file upload & listing, only an administrator can delete a decision

The suite specifically verifies permission boundaries — for example, that an employee cannot update a decision created by someone else, and that only an administrator can delete a decision — not just the “happy path” of each feature.

The frontend was verified with a production vite build, which completes without errors, and was manually exercised end-to-end against a live backend and PostgreSQL instance: registering a user, logging in, creating a decision, adding an alternative, posting a comment, and uploading a file all confirmed working.

9. Screenshots

The following screenshots were captured from the application running locally, backend served by Uvicorn against a real PostgreSQL database and frontend served by Vite — not mockups.

Login

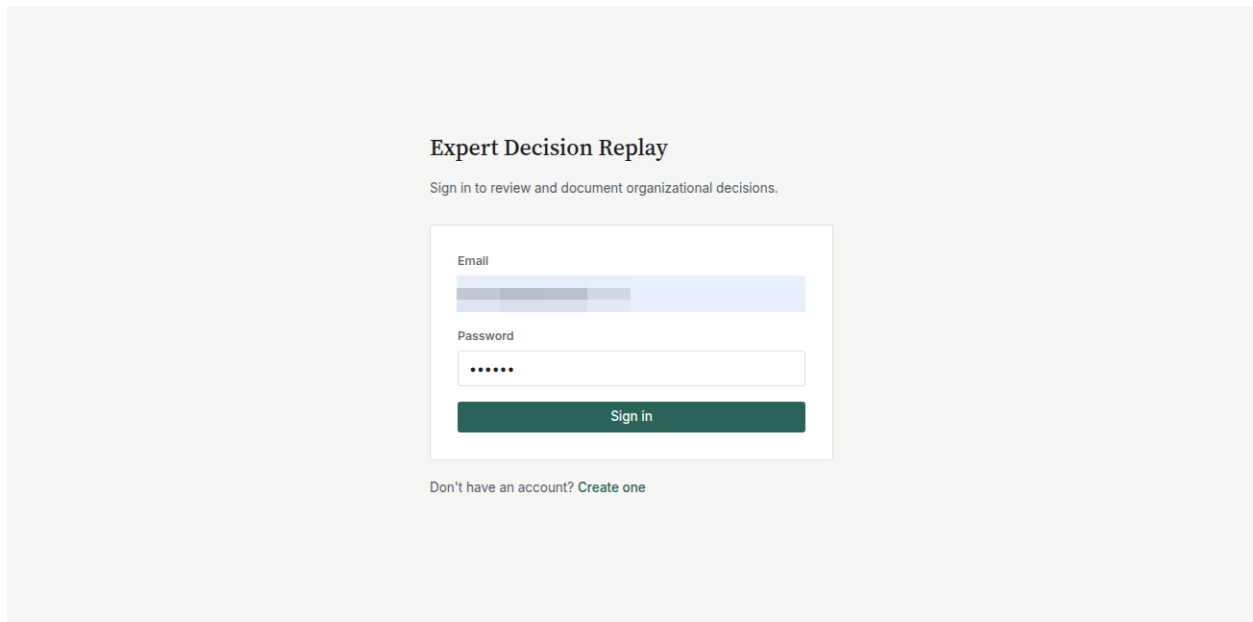


Figure 1: Login screen

Decisions list

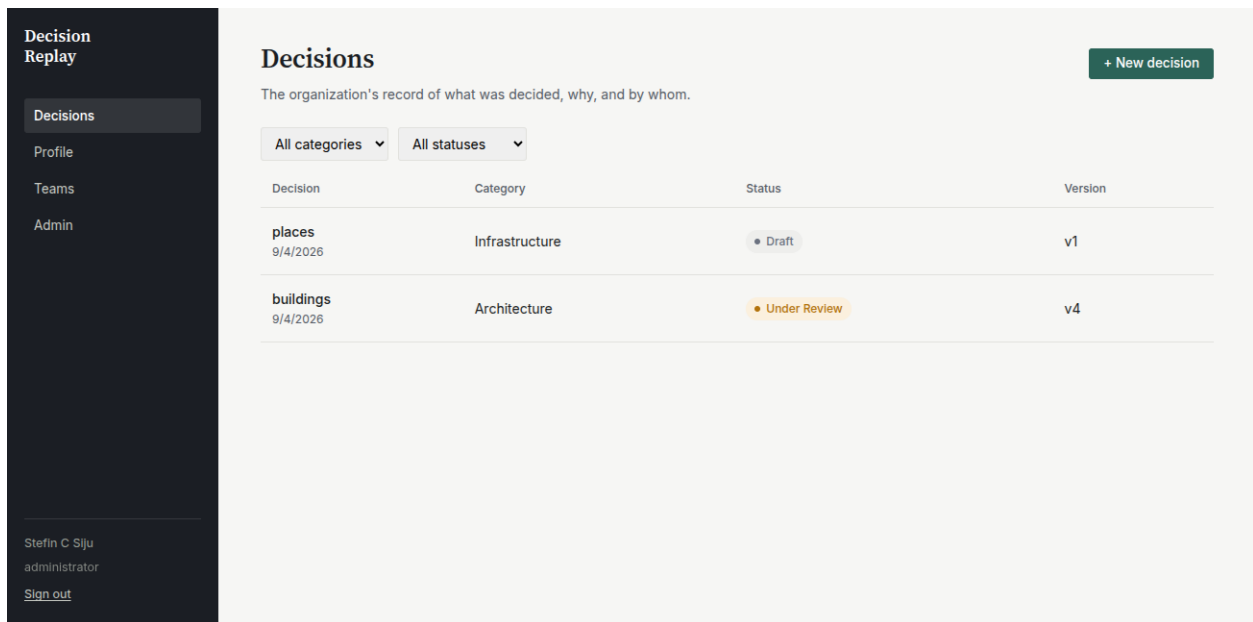


Figure 2: Decisions list

Alternatives, with live composite scoring

Two alternatives attached to the same decision, each with a different cost/risk/feasibility combination. The system computed both composite scores and automatically tagged “college” as **Recommended** since it scored higher (−10000) than “hospitals”

(−10002), despite “hospitals” having a lower estimated cost input weight — illustrating the scoring formula described in Section 7.3 in action, not just in theory.

The screenshot shows a web application interface for decision-making. On the left is a dark sidebar with the 'Admin' link at the top and the user 'Stefin C Siju administrator' with a 'Sign out' link at the bottom. The main content area has a 'Problem statement' box containing 'need for money'. Below this are tabs for 'Alternatives', 'Discussion', 'Files', and 'Version history', with 'Alternatives' selected. It shows '2 options considered' and a '+ Add alternative' button. The first alternative is 'college', marked 'Recommended', with 'Pros: students' and 'Cons: no of students'. Its scores are Cost: 10000000, Risk: 5/10, Feasibility: 5/10, and Composite: -10000. The second alternative is 'hospitals', with 'Pros: good for patients' and 'Cons: expensive'. Its scores are Cost: 10000000, Risk: 6/10, Feasibility: 4/10, and Composite: -10002.

Figure 3: Alternatives with composite scoring

Discussion (threaded comments)

The screenshot shows the 'Discussion' tab of the same application. The sidebar is identical. The main content area shows the 'Problem statement' box with 'need for money'. The 'Discussion' tab is selected, showing '2 comments'. The first comment by 'Stefin C Siju' 13h ago says 'hi' and has a 'Delete' button. The second comment by 'Stefin C Siju' 13h ago says 'need more ideas' and also has a 'Delete' button. At the bottom is a text input field 'Add a note, rationale, or question...' and a 'Post' button. The top of the main area shows 'Architecture · v4', the title 'buildings', a status 'Under Review' with a 'Change status...' dropdown, and a 'Delete decision' button.

Figure 4: Discussion tab

File attachments

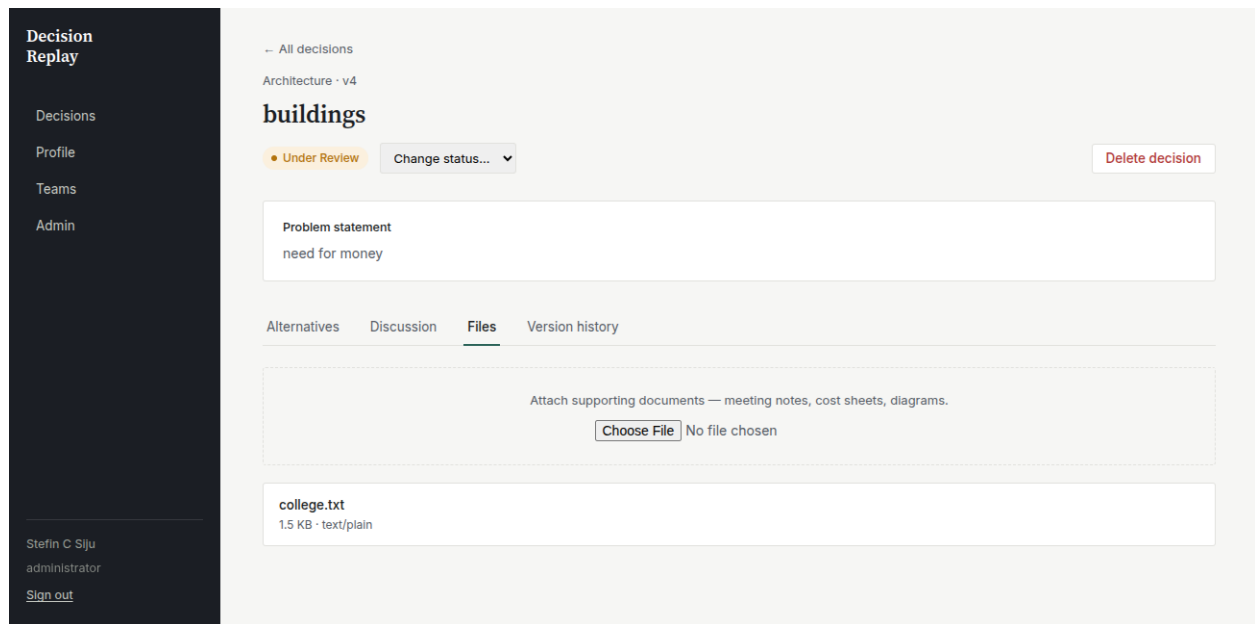


Figure 5: Files tab

Version history

Every status change on this decision produced an immutable version snapshot — four versions recorded from initial creation through three status changes.

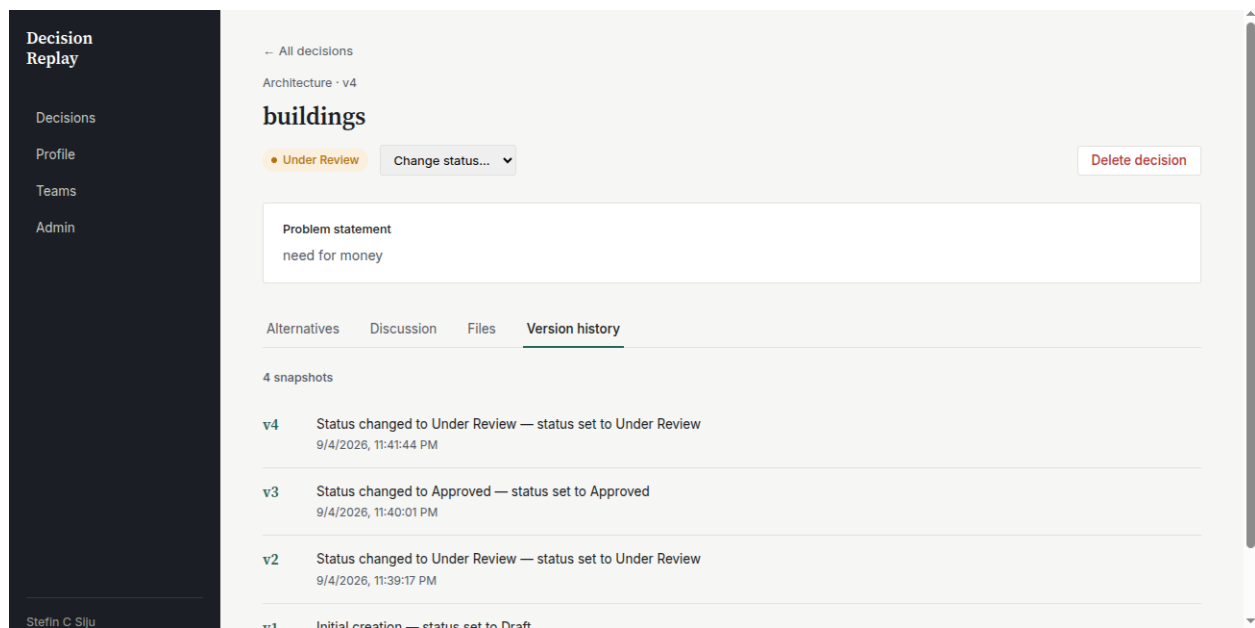


Figure 6: Version history tab

Interactive API documentation (Swagger UI)

Auto-generated by FastAPI at /docs, confirming every endpoint listed in Sections 6.2

and 7.2 is live and callable.

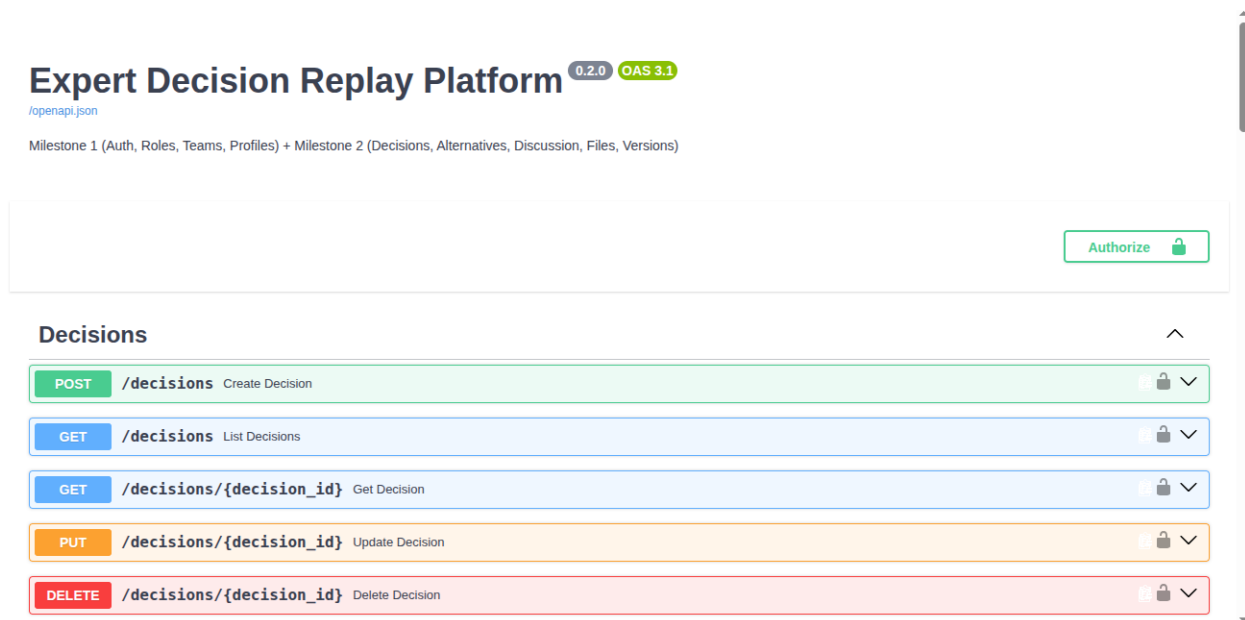


Figure 7: Swagger UI

10. Project Structure

```
edrp2/
├── Backend/
│   ├── models/          (Role, Team, User, UserProfile, Decision, Alternative,
│   │                   Comment, Attachment, DecisionVersion)
│   ├── Schemas/        (Pydantic request/response models)
│   ├── routers/          (decisions, alternatives, comments, attachments, versions)
│   ├── security/         (password hashing, JWT, auth dependencies)
│   ├── database/         (SQLAlchemy engine + session)
│   ├── tests/            (pytest suite, Milestone 1 + 2)
│   ├── main.py
│   └── requirements.txt
├── frontend/
│   └── src/
│       ├── pages/        (Login, Register, Profile, Teams, Admin,
│       │                DecisionsList, DecisionDetail)
│       └── components/    (Sidebar, tabs for Alternatives/Comments/Attachments/Versions)
├── docs/
│   ├── database-design.md (ERD + table definitions)
│   └── wireframes/        (low-fidelity screen wireframes, Milestone 1 & 2)
```

11. Conclusion & Next Steps

Milestones 1 and 2 together deliver a working decision-documentation platform: secure role-based authentication, team management, and a full decision lifecycle with alternatives, discussion, attachments, and a replayable version history — all backed by an automated test suite.

The schema and routing structure were built to extend cleanly, which sets up Milestone 3's planned scope: an analytics dashboard summarizing decisions by status, team activity feeds, and organization-wide reporting.